

International Islamic University Chittagong (IIUC)

Faculty of Science & Engineering

Department of Computer Science & Engineering (CSE)

PROJECT REPORT

Hybrid Multi-Level Queue CPU Scheduling Simulator

Course Information

Course Code: CSE-3632

Course Title: Operating System

Submitted By

Name	Student ID
Mohammed Shafiullah Raihan	C191092R
Mahi B Rahaman	C231161
M.M. Sadekul Islam	C231152
Tanvir Mahtab Parvez Sakib	C231141

Submitted To

MD. Alam Miah (Adj Faculty)

Department of Computer Science & Engineering

International Islamic University Chittagong (IIUC)

Date of Submission

18 December 2025

1. INTRODUCTION

CPU scheduling is a central topic in Operating Systems, responsible for deciding which process should run on the CPU at any given time. Different scheduling algorithms achieve different trade-offs between response time, throughput, fairness, and implementation complexity. Modern systems must handle a mix of workloads, such as interactive applications that require quick responses, and batch jobs that are less time-sensitive but may be long-running. This project implements a Hybrid MultiLevel Queue CPU Scheduling Simulator to demonstrate how multiple queues and scheduling policies can be combined to balance responsiveness for interactive processes and fairness for batch processes. The simulator uses a high-priority Round Robin queue for interactive tasks and a low-priority First-Come, First-Served queue for batch tasks, with strict preemption rules and a starvation mitigation mechanism based on aging. The simulator is implemented as a complete stack: a C++ core performs the actual scheduling simulation, a Node.js/Express backend coordinates communication, and a web-based frontend provides an interactive visual interface. The work aims to serve as a learning aid for students studying Operating Systems and CPU scheduling.

2. PROBLEM STATEMENT

A single scheduling algorithm often cannot satisfy all goals for all types of processes. For example, Round Robin is suitable for interactive workloads because it provides good response time, but it may be inefficient for long-running batch jobs. On the other hand, FCFS is simple and fair in order of arrival, but it can result in poor response time for interactive workloads. The problem addressed in this project is:

1. How to design and implement a hybrid scheduling system with multiple queues that handle different kinds of processes with different algorithms.
2. How to enforce a strict priority rule where interactive processes always preempt batch processes when necessary.
3. How to include an aging mechanism so that low-priority batch processes are not starved indefinitely.
4. How to present the scheduling behavior and metrics in an intuitive visual form suitable for teaching and experimentation.

The solution should be correct, configurable, and easy to experiment with.

3. SCHEDULING THEORY BACKGROUND

3.1 CPU Scheduling Basics

CPU scheduling determines which ready process should be assigned to the CPU. Common objectives include:

- Maximizing CPU utilization.
- Maximizing throughput (number of completed processes per unit time).
- Minimizing average waiting time and turnaround time.
- Providing acceptable response time for interactive users.

A scheduler typically works on a set of ready processes and uses some policy to choose the next process to run.

3.2 Multi-Level Queue Scheduling

In multi-level queue scheduling, processes are partitioned into separate queues based on some property, such as process type or priority class. Each queue can use its own scheduling algorithm. A strict or partial priority order is defined between queues, determining which queue is served first. For example, a system might have:

- A foreground queue for interactive tasks, using Round Robin.
- A background queue for batch tasks, using FCFS.

The scheduler always prefers the foreground queue when it is non-empty.

3.3 Round Robin (RR)

Round Robin scheduling assigns each process a small fixed CPU time slice called a time quantum. Processes are kept in a circular queue. The scheduler picks the process at the front, runs it for at most one quantum, and if it is not finished, moves it to the back of the queue. RR is preemptive and is designed to provide good average response time, particularly for interactive tasks, at the cost of more context switches.

3.4 First-Come, First-Served (FCFS)

First-Come, First-Served is the simplest scheduling algorithm. The process that arrives first is served first. Once a process starts running, it continues until completion (unless there is some external preemption rule). FCFS is fair in terms of arrival order but can suffer from the “convoy effect” where short jobs wait behind long ones, harming average waiting time and response time.

3.5 Starvation and Aging

Starvation occurs when a process never gets the CPU because other processes continually have higher priority. In strict-priority multi-level queue systems, low-priority queues may starve if high-priority queues are always busy. Aging is a common technique to prevent starvation. The priority of a process increases the longer it waits. Eventually, a long-waiting low-priority process is treated as a higher-priority one and gets CPU time.

4. SYSTEM OVERVIEW AND ARCHITECTURE

4.1 High-Level Architecture The project is structured into three layers:

- Frontend: A web-based interface where users define processes and parameters and view results.
- Backend: An Express server that exposes an HTTP API and orchestrates simulations.
- Core Engine: A C++ program that implements the scheduling logic and outputs JSON.

4.2 Components Overview

- C++ Engine (scheduler.cpp):

Implements the hybrid multi-level scheduling logic, including:

- Process data model.
- Two ready queues (Q1 for interactive, Q2 for batch).
- Dispatcher and preemption rules.
- Aging mechanism.
- Computation of metrics and JSON output.
- Backend (server.js):

A Node.js/Express server that:

- Accepts POST requests to /simulate.
- Validates and transforms request data.
- Writes a temporary text file in the format expected by the C++ engine.
- Executes the compiled C++ binary and captures its JSON output.

- Returns the JSON to the client.
- Frontend (public/index.html):

A single-page interface that:

- Lets users enter process definitions and configuration parameters.
- Sends simulation requests to the backend.
- Renders a Gantt-style timeline and tables of metrics.

4.3 Data Flow

1. User enters process data and configuration into the web UI.
2. UI sends a HTTP POST /simulate with a JSON payload.
3. Backend validates the payload, writes a temporary input file, and executes the C++ scheduler.
4. C++ engine reads the input file, runs the simulation, and prints a JSON object to stdout.
5. Backend parses the JSON and returns it as the HTTP response.
6. Frontend renders the timeline and metrics.

5. C++ SCHEDULING ENGINE DESIGN

5.1 Objectives and Requirements

The C++ engine must:

- Model each process with ID, arrival time, burst time, type, remaining time, waiting time, and turnaround time.
- Use Round Robin in a high-priority queue for interactive processes (type 0).
- Use FCFS in a low-priority queue for batch processes (type 1).
- Always choose a process from the high-priority queue when possible.
- Preempt a running batch process from Q2 after 1 time unit if there is at least one interactive process waiting in Q1.
- Implement an aging mechanism to promote long-waiting batch processes from Q2 to Q1.
- Produce a detailed timeline and metrics as a JSON object.

5.2 Data Structures and Process Model

Each process is represented by a structure with fields such as:

- `id`: unique identifier.
- `arrival_time`: arrival time of the process.
- `burst_time`: total CPU time required.
- `type`: 0 for interactive (Q1), 1 for batch (Q2).
- `remaining_time`: remaining CPU time as the simulation progresses.
- `completion_time`: time when process finishes.
- `waiting_time`: time spent waiting in queues.
- `turnaround_time`: total time from arrival to completion.
- `time_in_q2_wait`: cumulative waiting time in Q2, used for aging.

Schedulable processes are stored in a vector. Two deques are used as ready queues:

- Q1: holds indices of interactive processes (type 0).
- Q2: holds indices of batch processes (type 1).

The timeline is stored as a vector of entries (time, processId, queue).

5.3 Input Format and Configuration

The C++ program reads a simple text input either from a file (via command-line argument) or from standard input. Input format: `NRR_QUANTUM`

`AGING_THRESHOLD id1 arrival1 burst1 type1 ... idN arrivalN burstN typeN`

- `N` is the number of processes.
- `RR_QUANTUM` is the Round Robin time slice for Q1.
- `AGING_THRESHOLD` is the waiting time threshold in Q2 before promotion.
- Each process line defines one process.

If the config line is missing or invalid, default values such as `RR_QUANTUM = 2` and `AGING_THRESHOLD = 10` are used.

5.4 Queue Management and Dispatcher Logic

Processes are sorted by arrival time. At any given time:

- If Q1 is not empty, the scheduler selects the process at the front of Q1 and runs it with Round Robin.
- If Q1 is empty but Q2 is not, the scheduler selects the process at the front of Q2 and runs it with FCFS (subject to preemption).
- If both queues are empty, the CPU is idle.

For Round Robin (Q1):

- The chosen process receives up to `RR_QUANTUM` time units.
- If its remaining time is still positive after the quantum, it is moved to the back of Q1.

For FCFS (Q2):

- The chosen batch process runs until completion, unless it is preempted by a higher-priority process from Q1.

Preemption Rule:

- If a process from Q2 is currently running and at least one interactive process is present in Q1, then after one time unit of execution, the Q2 process is preempted.
- The preempted batch process is moved to the front of Q2.
- A counter of Q2 preemptions is maintained for reporting.

5.5 Starvation Mitigation (Aging Mechanism) To

avoid starvation of Q2 processes:

- Each time unit, for every process waiting in Q2 (but not currently running), its `time_in_q2_wait` is incremented.
- When `time_in_q2_wait` reaches or exceeds `AGING_THRESHOLD` and the process still has remaining time, the process is promoted to Q1:
- Its type is updated to 0 (interactive).
- It is pushed into Q1.
- A counter of aging promotions is incremented.

This ensures that a batch process that waits too long will eventually receive higher priority.

5.6 Metrics and Calculations For

each process:

- Completion Time: CT_i – time when the process finishes.
- Turnaround Time: $TAT_i = CT_i - AT_i$, where AT_i is arrival time.
- Waiting Time: $WT_i = TAT_i - BT_i$, where BT_i is burst time.

Aggregates:

- Average Waiting Time (AWT) = $(\sum WT_i) / N$.
- Average Turnaround Time (ATAT) = $(\sum TAT_i) / N$.
- CPU Utilization = $(\text{Total Busy Time}) / (\text{Total Time Span})$.
- Throughput = $(\text{Number of Completed Processes}) / (\text{Total Time Span})$.
- Aging Promotions: total count of $Q2 \rightarrow Q1$ promotions.
- Preemptions From Q2: total count of preemptions of Q2 processes due to Q1.

5.7 JSON Output Specification

The C++ program outputs a single JSON object that includes:

- config (effective rrQuantum, agingThreshold).
- timeline: array of entries { time, processId, queue }.
- processes: array with per-process metrics: id, arrivalTime, burstTime, type, waitingTime, turnaroundTime, completionTime.
- averageWaitingTime
- averageTurnaroundTime
- cpuUtilization
- throughput
- agingPromotions
- preemptionsFromQ2

This JSON is easy to parse in the backend and to consume in the frontend.

6. NODE.JS / EXPRESS BACKEND DESIGN

6.1 Responsibilities

The Express backend:

- Receives HTTP requests from the UI.
- Validates process lists and config parameters.
- Writes an input file for the C++ engine.
- Executes the compiled C++ scheduler binary.
- Parses the JSON result and returns it to the frontend.

6.2 API Contract (simulate)

Endpoint: POST /simulate. Preferred request body format: {"processes": [{ "id": 1, "arrivalTime": 0, "burstTime": 5, "type": 0 }, { "id": 2, "arrivalTime": 1, "burstTime": 3, "type": 1 }], "config": { "rrQuantum": 2, "agingThreshold": 10 }} The backend also supports a simpler legacy format where the body is just an array of processes. In that case default config values are used. The response is the JSON object generated by the C++ engine.

6.3 Error Handling and Validation

The backend checks that:

- The request body includes a valid processes array.
- Each process has numeric id, arrivalTime, burstTime, and type.

Invalid inputs result in an HTTP 400 response with an error message. If the C++ binary fails or returns invalid JSON, the server responds with HTTP 500 and logs details so that issues can be diagnosed.

7. WEB FRONTEND DESIGN (SIMULATION

WEBSITE) 7.1 UI Layout and Components

The

main web page contains:

- A header with the project title and a legend for queue colors (Q1, Q2, idle).
- A “Process Configuration” section where users can:
 - Add and remove process rows.
 - Edit ID, arrival time, burst time, and type.
 - Set Round Robin quantum and aging threshold.
 - Trigger the simulation.
- An “Execution Timeline and Metrics” section:

- Displays the Gantt chart.
- Shows average waiting time and turnaround time.
- Shows a per-process metrics table.

7.2 Gantt Chart Visualization The

Gantt chart:

- Has one cell per time unit.
- Each cell is colored and labeled to indicate:
- Which process is running (P1, P2, etc.).
- Which queue supplied the process (Q1 or Q2).
- Idle periods are indicated with a distinct style.
- A time axis is displayed under the chart to show the exact time units.

7.3 Interaction with Backend On

clicking “Run Simulation”:

1. JavaScript collects all process rows and scheduler parameters.
2. It builds the JSON request body and sends a POST /simulate.
3. When a successful response is received, it updates:
 - The Gantt chart using the timeline array.
 - The metrics panel using the averages and per-process metrics.
4. If any error occurs, an error message is displayed near the button.

8. USER MANUAL (HOW TO USE THE SIMULATOR)

8.1 Prerequisites

To run the simulator locally you need:

- A C++ compiler (such as g++).
- Node.js and npm installed on your system.

8.2 How to Run the Application Locally

1. Compile the C++ scheduler:

- On Windows (with g++ installed):

```
g++ scheduler.cpp -o scheduler.exe
```

- On Linux/macOS:

```
g++ scheduler.cpp -o scheduler
```

2. Install Node.js dependencies:

```
npm install
```

3. Start the server:

```
npm start
```

4. Open a web browser and visit:

<http://localhost:3000>

8.3 Using the Web Interface Step by Step

1. In the Process Configuration table, for each process enter:

- ID (for example 1, 2, 3).
- Arrival time (0 or greater).
- Burst time (1 or greater).
- Type (0 for interactive, 1 for batch).

2. Enter scheduling parameters:

- RR time quantum (for Q1).
- Aging threshold (in Q2 time units).

3. Click “Run Simulation” to start the simulation.

4. Wait for the results to be displayed. The timeline and metrics will update automatically.

8.4 Interpreting the Results

- Timeline:

Shows, for each time unit, which process ran and from which queue.

- Average metrics:
- Average Waiting Time indicates how long, on average, processes waited in queues.
- Average Turnaround Time measures total time from arrival to completion.
- Per-process metrics:

Show individual waiting time, turnaround time, and completion time for each process.

9. MATHEMATICAL FORMULAS AND EXAMPLE

9.1 Definitions Let:

- AT_i : Arrival time of process i .
- BT_i : Burst time of process i .
- CT_i : Completion time of process i .
- TAT_i : Turnaround time of process i .
- WT_i : Waiting time of process i .

9.2 Formulas

For each process:

- Turnaround Time: $TAT_i = CT_i - AT_i$
- Waiting Time: $WT_i = TAT_i - BT_i$
- Aggregate metrics:
- Average Waiting Time (AWT) = $(\sum WT_i) / N$
- Average Turnaround Time (ATAT) = $(\sum TAT_i) / N$

CPU Utilization:

- CPU Utilization = (Total Busy Time) / (Total Time Span)
- Throughput:
- Throughput = (Number of Completed Processes) / (Total Time Span)

9.3 Example Calculation Example processes:

- P1: Arrival = 0, Burst = 5
- P2: Arrival = 1, Burst = 3
- P3: Arrival = 2, Burst = 4

Assume after simulation we get:

- P1: $CT_1 = 7 \Rightarrow TAT_1 = 7 - 0 = 7$, $WT_1 = 7 - 5 = 2$
- P2: $CT_2 = 10 \Rightarrow TAT_2 = 10 - 1 = 9$, $WT_2 = 9 - 3 = 6$
- P3: $CT_3 = 12 \Rightarrow TAT_3 = 12 - 2 = 10$, $WT_3 = 10 - 4 = 6$ Then:
- $AWT = (2 + 6 + 6) / 3 = 14 / 3 \approx 4.67$
- $ATAT = (7 + 9 + 10) / 3 = 26 / 3 \approx 8.67$

10. TESTING AND VERIFICATION

10.1 Test Case 1 (Mixed Interactive and Batch)

Processes:

- P1: Arrival = 0, Burst = 5, Type = 0 (interactive, Q1)
- P2: Arrival = 1, Burst = 3, Type = 1 (batch, Q2)
- P3: Arrival = 2, Burst = 4, Type = 0 (interactive, Q1) Configuration:
- RR quantum = 2
- Aging threshold = 10

Expected behavior:

- P1 and P3, being interactive, get priority in Q1 and share the CPU in a Round Robin fashion.
- P2 runs in Q2 only when Q1 is empty.
- The final order and metrics can be checked manually for correctness.

10.2 Test Case 2 (Starvation and Aging) Processes:

- P1: Arrival = 0, Burst = 10, Type = 1 (batch, Q2)
- P2: Arrival = 1, Burst = 2, Type = 0 (interactive, Q1)
- P3: Arrival = 2, Burst = 2, Type = 0 (interactive, Q1)
- P4: Arrival = 3, Burst = 2, Type = 0 (interactive, Q1) Configuration:
- RR quantum = 2
- Aging threshold = 5

Expected behavior:

- P1 initially waits in Q2 while P2, P3, and P4 run in Q1.
- As time passes, P1's waiting time in Q2 increases.
- When P1's Q2 wait reaches the threshold, it is promoted to Q1 and eventually completes.
- The aging promotions counter should increase by at least one.

10.3 How to Reproduce the Tests To

reproduce these tests:

1. Run the application and open the web interface.
2. Enter the processes and configuration values as described.
3. Run the simulation.
4. Compare the resulting timeline and metrics with your expected values.

11. DEPLOYMENT CONSIDERATIONS

11.1 Local / Lab Deployment

The simplest deployment is a local setup on a lab machine or personal computer:

- Compile the C++ scheduler.
- Run the Node.js server.
- Access the web UI via <http://localhost:3000>.

This is sufficient for classroom demonstrations and assignments.

11.2 Separation of Frontend and Backend For

more advanced deployments:

- Host the static frontend on a static web host (or even on university web servers).
- Host the backend and C++ engine on a server/VPS or PaaS that supports Node.js and native binaries.
- Configure the frontend to send requests to the backend's public URL instead of localhost.

12. Usage and Applications

12.1 Target Audience

This simulator is meticulously designed to serve a broad spectrum of users in the academic and professional spheres:

- **Computer Science Students:**
 - Designed for those enrolled in **Operating Systems** courses who need to grasp fundamental CPU scheduling concepts.
 - Ideal for learning the complexities of **multi-level queue scheduling**, preemption, and starvation mitigation.
 - Serves as a vital study aid for exams and assignments involving scheduling algorithms.
- **Educators and Instructors:**
 - Provides professors with a tool to visually demonstrate dynamic scheduling behavior during lectures.
 - Assists teaching assistants in providing interactive lab sessions. ○ Helps course designers create practical exercises and graded assignments.
- **Researchers and Developers:**
 - Enables researchers to study the performance trade-offs of different scheduling algorithms.
 - Helps system programmers visualize the internal mechanics of **hybrid schedulers**. ○ Interested parties can observe how varied scheduling policies directly impact process execution.

12.2 Educational Use Cases

The primary educational applications focus on bridging the gap between theoretical OS concepts and practical visualization:

1. **Multi-Level Queue Scheduling:** Students observe how processes are categorized into distinct queues based on their functional type.
2. **Round Robin (RR) Algorithm:** Users can experiment with varied **time quantum** values to see their direct impact on response time and waiting times.

3. **FCFS Scheduling:** Demonstrates the handling of batch processes and allows for side-by-side comparison with Round Robin behavior.
4. **Preemption Concepts:** Provides a clear visual of when and why high-priority processes interrupt low-priority ones.
5. **Starvation and Aging:** Users can witness "starvation" when interactive processes dominate and observe how the **aging mechanism** promotes processes to prevent it.
6. **Performance Metrics:** Simplifies the learning of calculations for waiting time, turnaround time, CPU utilization, and throughput.

12.3 Practical Scenarios

The simulator can model several real-world computing environments:

Scenario	Interactive Processes (Type 1)	Batch Processes (Type 2)	Practical Use Case
Office System	Word processing Web browsing	Virus scan Backup	Ensuring UI responsiveness while background tasks complete.
Server Environment	User Monitoring Dashboards	Data processing Report generation	Balancing user-facing services with heavy back-end jobs.
Developer Workstation	IDE, Debugger	Compilation Analysis	Keeping developer tools responsive during long builds.
Research Cluster	Jupyter notebooks Data sessions	Simulations Parameter sweeps	Prioritizing interactive research over long-running jobs.

12.4 Benefits and Advantages

- **Visual Learning:** The **Gantt chart** and color-coded execution patterns make abstract concepts concrete.
- **Interactive Experimentation:** Replaces tedious manual tracing on paper with instant, digital results.

- **Comprehensive Metrics:** Features automatic calculation of AWT and ATAT with per-process breakdowns to identify fairness issues.
- **Realistic Implementation:** The C++ engine shows how theoretical algorithms translate into production-quality code.
- **Configurable Parameters:** Users can fine-tune the RR quantum and aging threshold to test boundary conditions.

12.5 Step-by-Step Usage Workflow

1. **Setup:**
 - o Compile the C++ scheduler: `g++ scheduler.cpp -o scheduler.exe`
 - o Install Node.js dependencies: `npm install`
 - o Start the server: `npm start`
 - o Access the dashboard: `http://localhost:3000`
2. **Define Workload:** Input process IDs, arrival times, and burst times. Assign types (Interactive or Batch).
3. **Run Simulation:** Click the **"Run Simulation"** button to trigger the C++ core.
4. **Analyze Results:** Examine the Gantt chart and review the calculated performance metrics.
5. **Experiment:** Modify the quantum or aging threshold and re-run to compare behavioral shifts.

12.6 Comparison with Other Tools

- **vs. Manual Calculation:** Faster, eliminates human error, and is significantly more scalable for large process sets.
- **vs. Text-Based Simulators:** The web-based Gantt chart is more intuitive and accessible than raw command-line output.
- **vs. Complex OS Simulators:** This tool is specifically focused on MLQ, making it easier to understand without "complexity overload."

12.7 Real-World Applications

This simulator models logic found in high-level systems:

- **Linux Kernel:** Uses multi-level scheduling policies for different process classes.
- **Windows Task Scheduler:** Prioritizes foreground applications over system background tasks.
- **Cloud Computing:** Schedulers prioritize user-facing requests over internal batch maintenance.
- **Embedded & Database Systems:** Prioritize critical or interactive queries over non-critical operations.

12.8 Limitations and Scope

To maintain educational focus, the simulator intentionally excludes:

- **I/O Operations:** Does not model processes blocking for disk or network I/O.
- **Multiple CPUs:** Limited to single-CPU scheduling simulations.
- **Memory Management:** Does not account for RAM constraints or swapping.
- **Dynamic Creation:** Processes are static and predefined for the simulation run.

13. CONCLUSION AND FUTURE WORK

This project implements a Hybrid Multi-Level Queue CPU Scheduling Simulator that combines a Round Robin queue for interactive processes with a FCFS queue for batch processes, plus preemption and aging. It demonstrates how operating systems can prioritize interactive workloads while still ensuring that batch jobs eventually complete. The simulator provides detailed metrics and a visual timeline, making it an effective teaching tool. Possible future enhancements include:

- Adding more scheduling algorithms such as Shortest Job First (SJF), Shortest Remaining Time First (SRTF), or static/dynamic priority scheduling.
- Implementing multi-level feedback queues with dynamic adjustment of process priority based on behavior.
- Modeling I/O-bound processes and blocking states for a more realistic system.
- Adding more advanced analytic views and export options for research and coursework.

14. REFERENCES

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, “Operating System Concepts,” Wiley.
2. Andrew S. Tanenbaum and Herbert Bos, “Modern Operating Systems,” Pearson.
3. University lecture notes and slides on CPU scheduling and multi-level queue scheduling.
4. Official Node.js and Express documentation.